Forma Dev Web
+ DevOps

_hdrien

Développer un service et l'héberger

01
Frontend
HTML, CSS, Javascript

02
Backend
Exemples avec Python et Flask

03
Conteneurisation
Docker, c'est quoi ?

04
CI/CD
GitLab & pipelines

05
Déploiement
Kubernetes & Rancher

06
Aller plus loin
Multi-container apps, bases de données, persistence

Frontend

Trois solutions

L'écrire soi-même

+ 100% personnalisé
- C'est long
- Pas forcément facile

Partir d'un template

+ Rapide et personnalisable
- Faut trouver le template

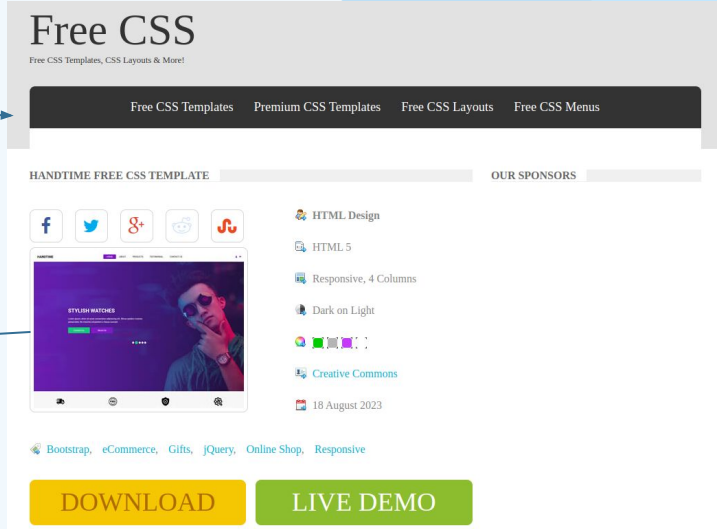
Voler son frontend

+ Gratuit
- Pas très propre

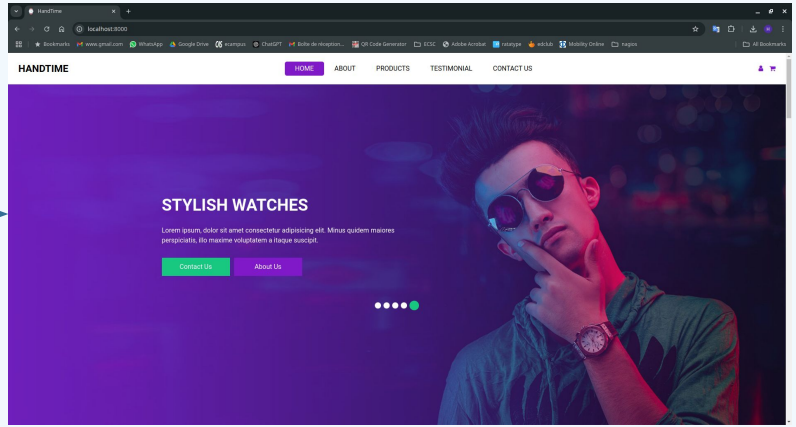
Free CSS Templates
https://www.free-css.com › free-css-templates

Download 3552 Free Website Templates - CSS & HTML

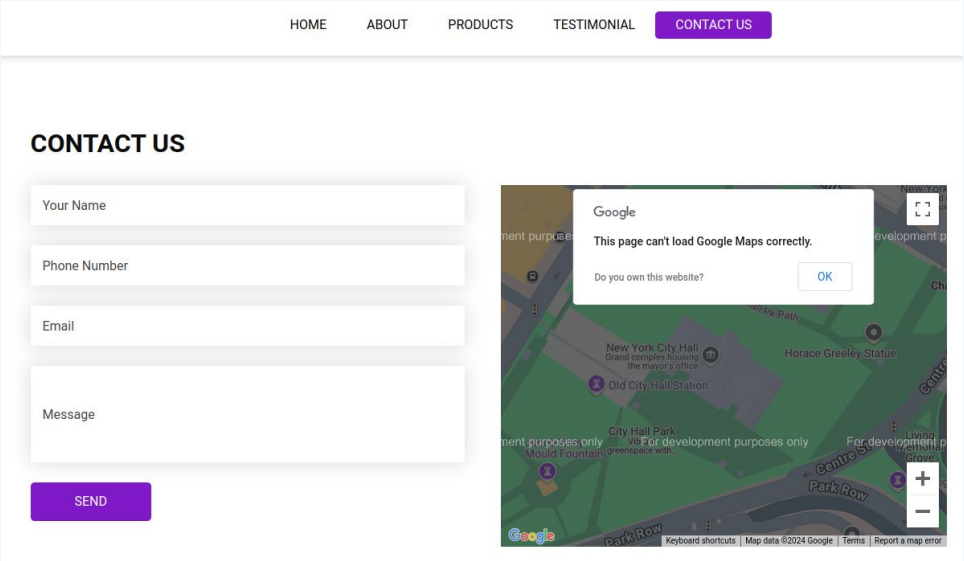
The **free** website **templates** that are showcased here are open source, creative commons or totally **free**. These **free** CSS **HTML templates** can be freely ...



```
>>> handtime-html python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

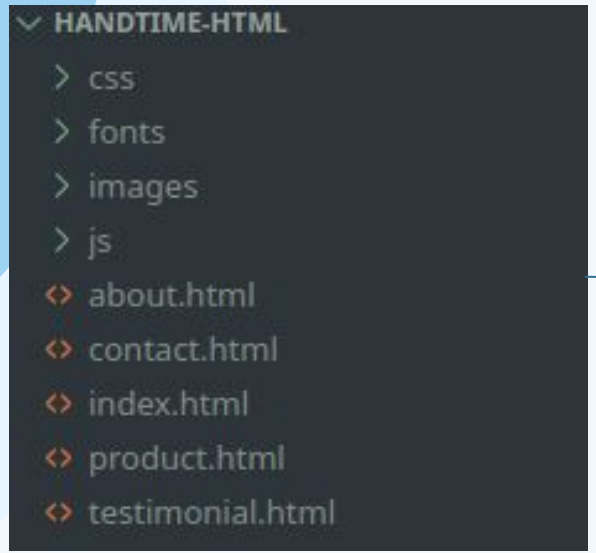


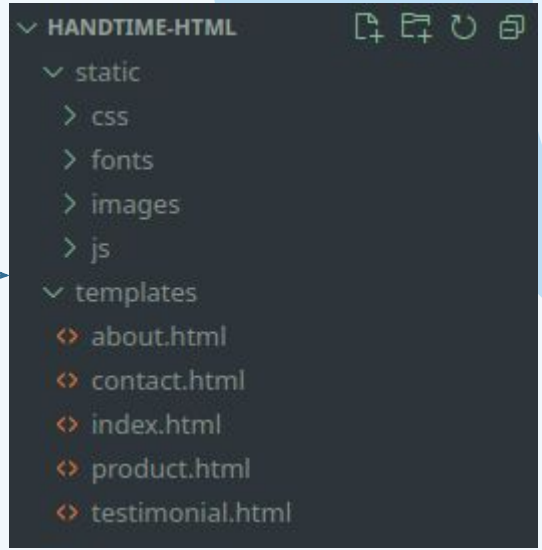
On veut un site dynamique



On va utiliser Flask pour faire marcher cette page !

1 - On restructure le projet





2 - On convertit les liens vers les ressources statiques

```
<!-- font awesome style -->
<link href="css/font-awesome.min.css" rel="stylesheet" />
```

```
<!-- font awesome style -->
<link href="{{ url_for('static', filename='css/font-awesome.min.css') }}" rel="stylesheet" />
```

Ça s'automatise, voir le fichier *urlfor-converter.py* sur le repo
https://gitlab.hackademint.org/tp-infra/tp-devops

3 - On crée nos routes

```python
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/contact', methods=['GET'])
def contact():
    return render_template('contact.html')

@app.route('/contact', methods=['POST'])
def submit_contact():

    message = "Votre demande a bien été envoyée."

    if request.form['message'] == 'mot de passe':
        message = "Bienvenue. La réponse que tu cherches est hackademint."

    return render_template('contact.html', message=message)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, threaded=True, debug=True)
```

4 - On ajoute la partie dynamique grâce au templating (Jinja)

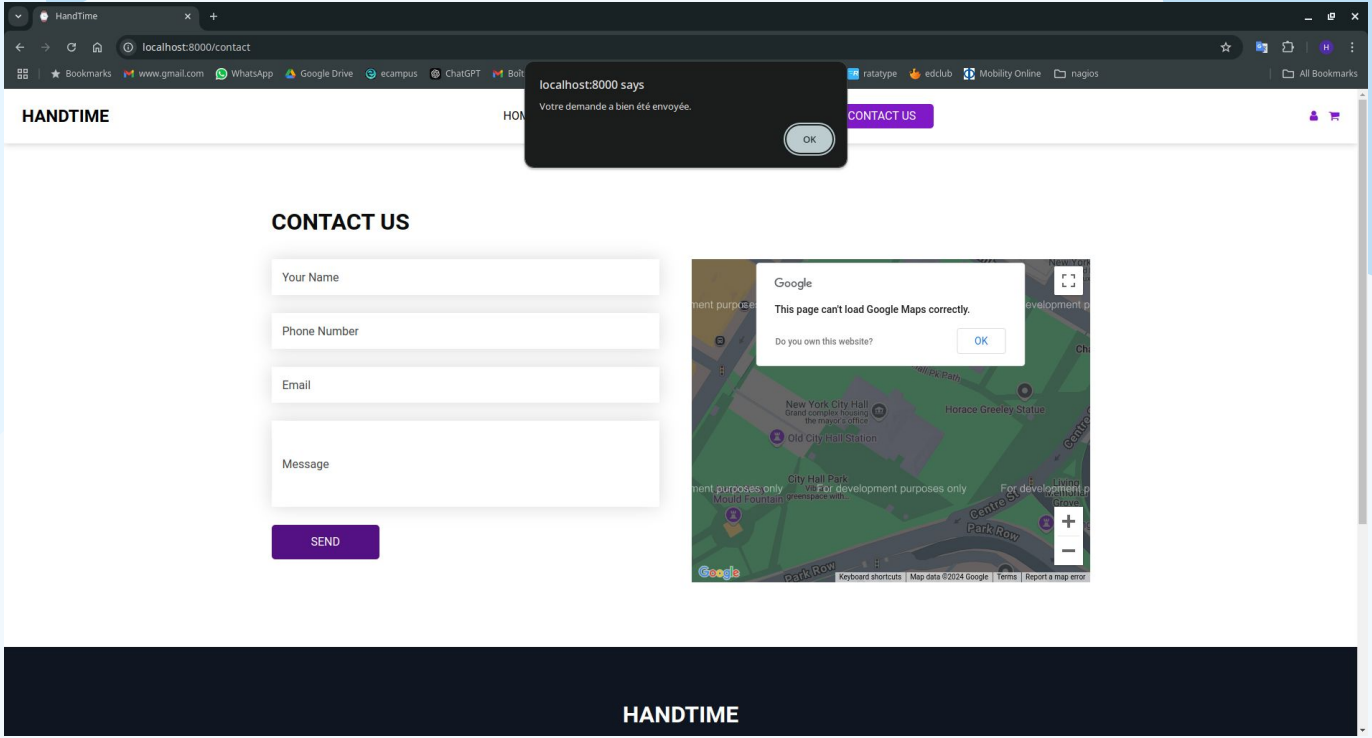
```
  <script type="text/javascript" src="{{ url_for('static', filename='js/custom.js') }}"></script>
  <!-- Google Map -->
  <script src="https://maps.googleapis.com/maps/api/js?key=AIzaSyCh39n5U-4IoWpsVGUHWdqB6puEkhRLdmI&callback=myMap">
  </script>
  <!-- End Google Map -->

  {% if message %}
  <script>
    alert("{{ message }}");
  </script>
  {% endif %}


</body>

</html>
```

À la fin du fichier *templates/contact.html*.
(Voir la slide précédente pour comprendre d'où sort message)

5 - On lance le serveur

```
>>> handtime git:(main) ✗ source venv/bin/activate
(venv) >>> handtime git:(main) ✗ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a producti
on WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://192.168.0.10:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 482-627-257
```

On a notre site dynamique !



Conteneurisation

Ça marche sur ~~ma machine~~ toutes les machines !

Je vais pas détailler (allez regarder sur internet pour + d'infos) mais **conteneurisation = mettre son app et tout ce dont elle a besoin (dépendances mais aussi OS) dans un package (= un conteneur)**. Une fois l'app conteneurisée elle marchera pareil partout.

Dockerfile

La recette du conteneur

handtime > Dockerfile > COPY

```
1   FROM python:3-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt /app
6
7   RUN pip install --no-cache-dir -r requirements.txt
8
9   COPY . /app
10
11  EXPOSE 8000
12
13  CMD ["python", "app.py"]
```

On oublie pas d'exporter la liste des dépendances python à l'avance avec ***pip freeze > requirements.txt*** !

Image du conteneur

On construit l'image décrite par la recette

```
(venv) >>> handtime git:(main) ✗ sudo docker build -t monimage .
[sudo] password for hdrien:
[+] Building 1.1s (10/10) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 203B                                          0.0s
 => [internal] load metadata for docker.io/library/python:3-slim              0.9s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [internal] load build context                                             0.1s
 => => transferring context: 126.54kB                                         0.1s
 => [1/5] FROM docker.io/library/python:3-slim@sha256:4efa69bf17cfbd83a9942e60e2642335c3  0.0s
 => CACHED [2/5] WORKDIR /app                                                 0.0s
 => CACHED [3/5] COPY requirements.txt /app                                   0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt           0.0s
 => CACHED [5/5] COPY . /app                                                  0.0s
 => exporting to image                                                        0.0s
 => => exporting layers                                                       0.0s
 => => writing image sha256:5c14092a2d45a290f272219acb857ed0066f520a7268051b344546d42d76  0.0s
 => => naming to docker.io/library/monimage                                   0.0s
(venv) >>> handtime git:(main) ✗
```

Finalement, le conteneur !

On instancie l'image qu'on a construite

```
(venv) >>> handtime git:(main) ✗ sudo docker run -it -p 80:8000 --name monconteneur monimage
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://172.17.0.2:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 806-421-424
```

Le *-p 80:8000* signifie qu'on lie le port 8000 du conteneur avec le port 80 de notre machine -> **on peut acceder au site en tapant <u>http://localhost</u> sur notre navigateur !**

GitLab

Triplement utile ici

https://gitlab.hackademint.org

Version control

- Héberge notre repo git
- Permet de collaborer avec d'autres personnes

CI / CD

- Permet de définir des actions automatisées (par exemple à chaque push)
- Ex : peut build nos images conteneur à notre place (voir slide d'après)

Registry

- Héberge les images qu'il a construit pour nous

Rappel

L'exemple complet est dispo sur

https://gitlab.hackademint.org/tp-infra/tp-devops

CIC/CD GitLab

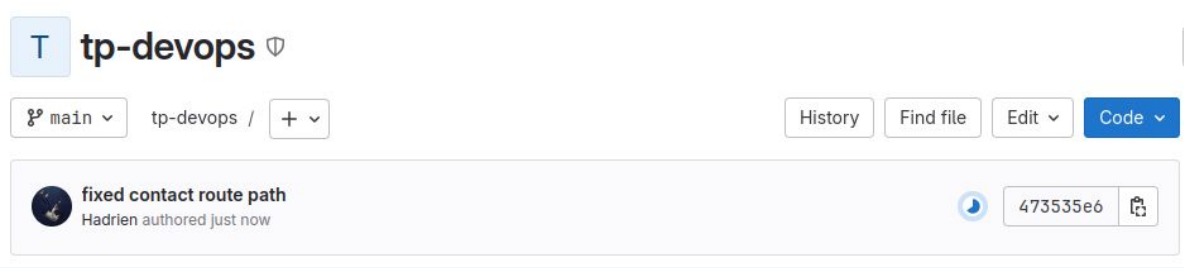
Le fichier *.gitlab-ci.yml*

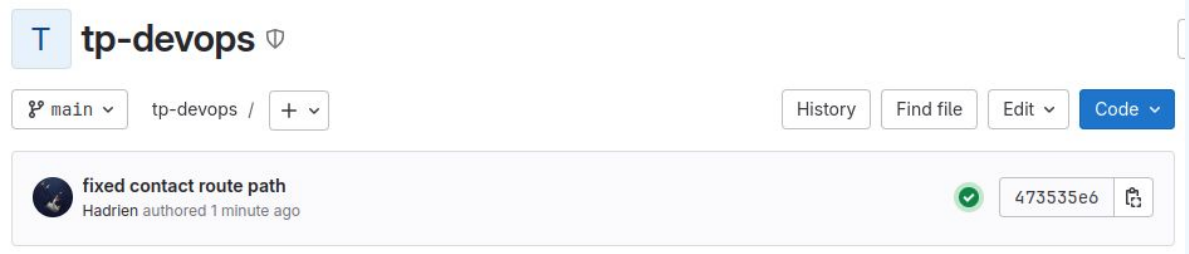
```
docker:
  stage: containerize
  image:
    name: gcr.io/kaniko-project/executor:debug
    entrypoint: [""]
  script:
    - mkdir -p /kaniko/.docker
    - echo "{\"auths\":{\"${CI_REGISTRY}\":{\"auth\":\"$(printf "%s:%s" "${CI_REGISTRY
    - >-
      /kaniko/executor
      --context "${CI_PROJECT_DIR}/handtime"
      --dockerfile "${CI_PROJECT_DIR}/handtime/Dockerfile"
      --cache=true
      --cache-ttl=72h
      --destination "${CI_REGISTRY_IMAGE}"
```

À part ces deux lignes vous avez rien besoin de changer

Ici on dit à GitLab de build l'image du conteneur (comme on a fait tout à l'heure, mais cette fois c'est automatisé et fait à chaque push) + de mettre l'image sur notre registry

CIC/CD GitLab



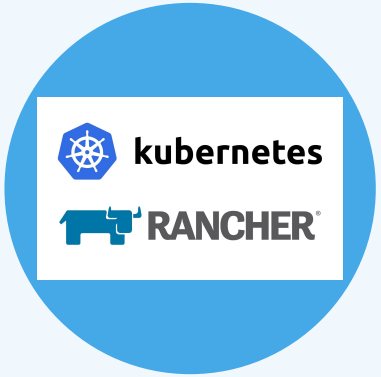


Vous pouvez retrouver les détails dans l'onglet *Build* > ***Jobs*** du repo

CIC/CD GitLab

```
77  INFO[0009] Taking snapshot of full filesystem...
78  INFO[0009] Pushing layer registry.hackademint.org/tp-infra/tp-devops/cache:ab1deab35dc69816b49c2345451568fbc9092838ec6e6f0600a9cd728cbb35f0 to cache now
79  INFO[0009] Pushing image to registry.hackademint.org/tp-infra/tp-devops/cache:ab1deab35dc69816b49c2345451568fbc9092838ec6e6f0600a9cd728cbb35f0
80  INFO[0010] COPY . /app
81  INFO[0010] Taking snapshot of files...
82  INFO[0010] EXPOSE 8000
83  INFO[0010] Cmd: EXPOSE
84  INFO[0010] Adding exposed port: 8000/tcp
85  INFO[0010] No files changed in this command, skipping snapshotting.
86  INFO[0010] CMD ["python", "app.py"]
87  INFO[0010] No files changed in this command, skipping snapshotting.
88  INFO[0012] Pushed registry.hackademint.org/tp-infra/tp-devops/cache@sha256:dd6387ffdffb62f9d068a313165e5dda13b46bfd1af8471bbc8fd385d6f3454f
89  INFO[0012] Pushing image to registry.hackademint.org/tp-infra/tp-devops
90  INFO[0016] Pushed registry.hackademint.org/tp-infra/tp-devops@sha256:159098b7f14af767ba965d7707f85b8feabe2abe324b0688f7e192cd0d19bf8d
91  Cleaning up project directory and file based variables                                                                          00:01
92  Job succeeded
```

Une fois le *Job* fini l'image est publiée sur le *registry* hackademint - les détails sont dans l'onglet *Deploy* > ***Container Registry*** du repo



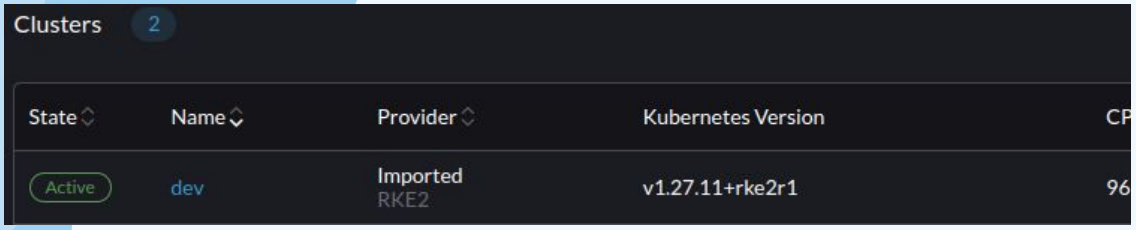
Déploiement

C'est l'heure de déployer votre site web sur l'infra HackademINT à l'aide de Kubernetes et Rancher !

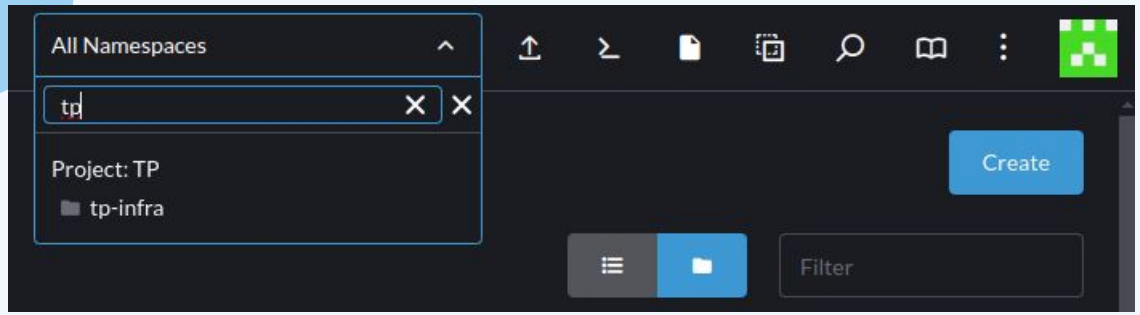
https://rancher-prod.priv..hackademint.org

Déploiement

Créer un déploiement



On sélectionne la dev



On sélectionne le bon namespace

Déploiement

Ensuite *Workloads* > *Deployments* puis bouton “Create”

Project: TP

Deployment: tp-devops Active

Namespace: tp-infra Age: 6 days Pod Restarts: 0

Namespace *
tp-infra

Name *
tp-devops

Nom

Description
Any text you want that better describes this resource

Replicas *
1

Deployment Pod web + Add Container

General
Health Check
Resources
Security Context
Storage

General

Container Name
web

Nom du conteneur

Init Container
Standard Container

Image

Container Image
registry.hackademint.org/tp-infra/tp-devops:latest

URL de l’image

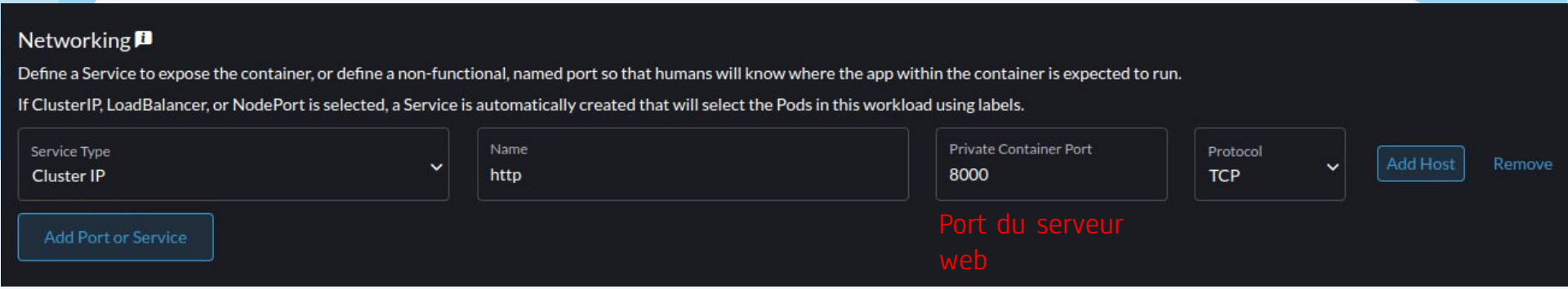
Pull Policy
Always

Pull Secrets
gitlab-pull-token

Secret pour accéder au registry (existe sûrement déjà sur le namespace, il faut juste le sélectionner

Déploiement

Ajouter un service dans “Networking” pour pouvoir lier le conteneur à un *ingress*



La config du déploiement est finie, on peut appuyer sur “Create”

Ingress

On va lier un sous domaine d'*hackademint.org* à notre conteneur serveur web. Onglet *Service Discovery* > ***Ingresses*** puis bouton "Create"

Ingress: tp-devops-ingress Active

Namespace: tp-infra Age: 6 days

Ingresses route incoming traffic from the internet to Services within the cluster based on the hostname and path specified in the request. You can expose multiple Services on the same external IP address and port.

Namespace * tp-infra

Name * tp-devops-ingress

Description Any text you want that better describes this resource

Rules

Default Backend

Certificates

Ingress Class

Labels & Annotations

Rules

Request Host devops.dev.hackademint.org

Path: Prefix /

Target Service: tp-devops

Port: 8000

Remove

Add Path

Add Rule

C’est fini !

Le service est déployé sur l’infra d’HackademINT
et accessible depuis Internet !